

DevSecOps CI/CD Pipeline Project Report

Field	Details
Project Title	Secure Enterprise DevSecOps CI/CD Pipeline using GitHub Actions, Vault, Terraform & Docker
Project Name	Project 5 — DevSecOps CI/CD Pipeline
Submitted By	Lukman Ali
Assessment Type	DevSecOps CI/CD Pipeline Implementation
Target Architecture	Secure AWS-Based Automated CI/CD Infrastructure
Academic Domain	DevOps, DevSecOps & Cloud Computing
Batch	Cloud-T1-Haldia Institute Of Technology
GitHub Repo Link	https://github.com/lukmanali504/devsecops-ci-cd-pipeline.git

1. Introduction

Modern software development requires faster delivery cycles while maintaining strong security practices. Traditional development models often perform security testing only during the later stages of deployment, which can lead to increased vulnerabilities, delayed releases, and higher remediation costs.

DevSecOps addresses this challenge by integrating security directly into the DevOps lifecycle. It ensures that security checks, vulnerability assessments, and compliance validations are performed continuously throughout the software delivery pipeline.

This project demonstrates the implementation of a complete DevSecOps CI/CD pipeline using industry-standard tools and cloud technologies. The pipeline automates application building, security scanning, infrastructure provisioning, containerization, and deployment across multiple environments.

The entire workflow was implemented in a real AWS cloud environment using automated CI/CD execution with integrated security controls.

The implemented pipeline performs the following tasks automatically:

- Builds the application
- Performs multiple security scans
- Provisions infrastructure using Terraform
- Retrieves secrets securely from HashiCorp Vault
- Builds and pushes Docker images
- Deploys containers automatically
- Supports Dev, Staging, and Production environments

The project provides practical exposure to real-world DevSecOps implementation and secure cloud-native deployment practices.

2. Objectives

The primary objective of this project was to design and implement a secure and automated DevSecOps pipeline capable of managing application deployment from development to production.

The project aims to:

- Implement a secure DevSecOps CI/CD workflow
- Automate build, testing, and deployment processes
- Integrate security scanning tools into the CI/CD pipeline
- Provision cloud infrastructure using Terraform
- Manage sensitive credentials securely using HashiCorp Vault
- Containerize applications using Docker
- Implement automated deployment using GitHub Actions
- Deploy applications across multiple environments
- Enforce security policy gates during deployment
- Gain practical experience with enterprise DevSecOps practices

3. Technologies Used

The following technologies and tools were used to implement the DevSecOps pipeline.

Technology	Category	Purpose	Role in Project
GitHub	Version Control Platform	Source code management	Repository hosting, workflow storage, branch management, and CI/CD trigger source
GitHub Actions	CI/CD Automation	Pipeline automation	Executes security scans, Docker build, Terraform provisioning, and deployment workflows
AWS EC2	Cloud Infrastructure	Virtual server hosting	Hosts Management EC2, Vault server, and Deployment EC2 containers
Docker	Containerization Platform	Application containerization	Packages the Node.js application into portable containers
Docker Hub	Container Registry	Docker image storage	Stores and distributes Docker images for deployment
Terraform	Infrastructure as Code (IaC)	Infrastructure automation	Automatically provisions AWS EC2 instances, security groups, and networking resources
HashiCorp Vault	Secrets Management	Secure credential storage	Stores AWS credentials and DockerHub credentials securely
Trivy	Vulnerability Scanner	Dependency and container scanning	Detects vulnerabilities in Node.js dependencies, filesystem, and Docker images
Semgrep	SAST Security Tool	Static Application Security Testing	Detects insecure coding patterns, missing middleware, and Docker security issues
Gitleaks	Secret Scanning Tool	Secret leak detection	Detects hardcoded passwords, API keys, SSH keys, and tokens
Checkov	IaC Security Scanner	Infrastructure security validation	Scans Terraform and Dockerfile configurations for security misconfigurations

4. System Architecture

Architecture Workflow

The following workflow represents the complete DevSecOps pipeline architecture implemented in this project.

```

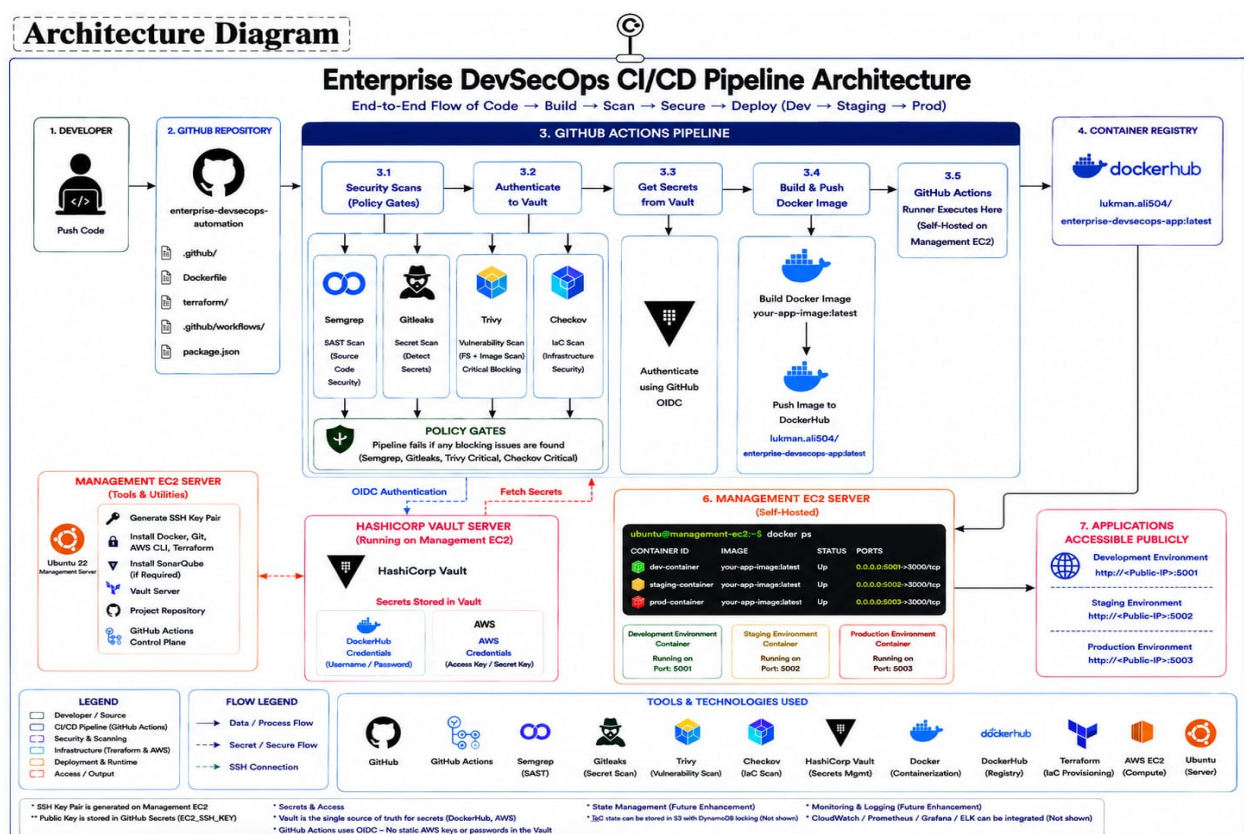
Developer
  ↓
GitHub Repository
  ↓
GitHub Actions Pipeline
  ├── Semgrep (SAST)
  ├── Gitleaks (Secret Scan)
  ├── Trivy (Vulnerability Scan)
  └── Checkov (IaC Scan)
  ↓
Authenticate to Vault
  ↓
Fetch AWS + DockerHub Credentials
  ↓
Build Docker Image
  ↓
Push Image to DockerHub
  ↓
Terraform Provisioning
  ↓

Deploy Dev / Staging / Prod Containers

  ↓
Applications Accessible Publicly

```

Architecture Diagram



5. Project Workflow

The project workflow was designed to automate secure application delivery through multiple CI/CD stages integrated with security validation mechanisms.

CI Pipeline Stages

1. Source code is fetched from the GitHub repository.
2. Security scans are executed using:
 - Gitleaks for secret detection
 - Semgrep for SAST analysis
 - Trivy for vulnerability scanning
 - Checkov for Infrastructure as Code security validation
3. Docker image is built using secure container configurations.
4. The Docker image is pushed to Docker Hub for centralized storage and deployment.

CD Pipeline Stages

1. I used a self-hosted GitHub Actions runner configured on my own EC2.
2. Updated Docker containers are deployed across multiple environments:
3.
 - Development Environment
 - Staging Environment
 - Production Environment

The complete workflow enables automated, secure, and consistent application deployment across all environments.

6. AWS Infrastructure Setup

Single AWS EC2 instance were used in this project.

6.1 Management EC2

The Management EC2 server was used as the central DevSecOps management system.

Purpose

- Vault setup and management
- Project development
- SSH key generation

Components Hosted

- HashiCorp Vault
- Terraform configurations
- Dockerfile
- Application source code
- GitHub workflow files

6.2 Management EC2

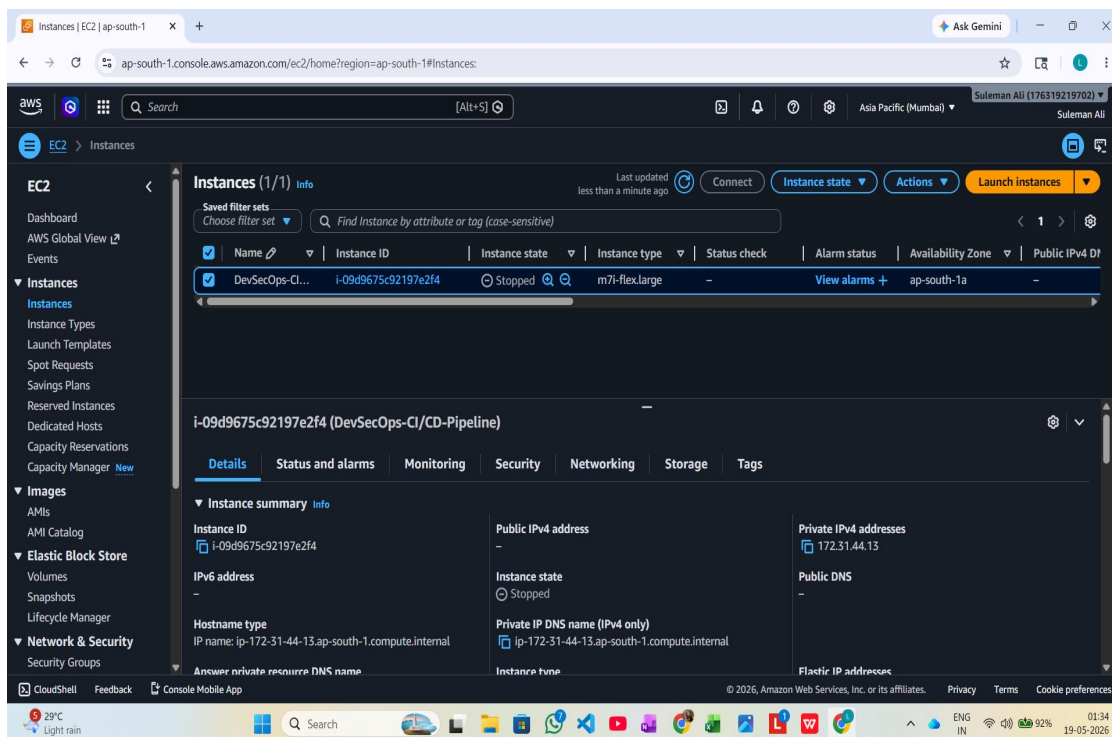
The Management self EC2 server configured using github actions-runner on my own EC2 server.

Purpose

- Hosting deployed containers
- Running Dev environment
- Running Staging environment
- Running Production environment

Deployment Ports

Container	Port
Dev Container	5001
Staging Container	5002
Production Container	5003



7. Implementation Flow

7.1 Application Development

A secure Python based application was developed and deployed inside Docker containers across Development, Staging, and Production environments. The application was designed with multiple

security middleware integrations to align with DevSecOps security practices and reduce common web application vulnerabilities.

7.2 Dockerization of Application

After application development, Docker was used to containerize the app.py application. Docker ensures consistent deployment across different environments by packaging the application along with its dependencies into portable containers.

A Dockerfile was created to define the application image configuration. The Docker container exposes port 5000 and executes the application automatically during runtime.

Containerization simplified deployment automation and enabled environment-based deployments across Development, Staging, and Production environments.

Dockerfile Features

- Python Slim base image
- Dependency installation
- Non-root user configuration
- Docker health checks
- Secure runtime configuration

```
1  FROM python:3.11-slim
2
3  WORKDIR /app
4
5  RUN adduser --disabled-password appuser
6
7  COPY requirements.txt .
8
9  RUN pip install --no-cache-dir -r requirements.txt
10
11 COPY . .
12
13 EXPOSE 5000
14
15 USER appuser
16
17 CMD ["python", "app.py"]
```

7.3 HashiCorp Vault and GitHub Secrets Management

HashiCorp Vault and GitHub Actions Secrets were used together to securely manage sensitive credentials required throughout the DevSecOps pipeline.

Vault was used as the centralized secrets management solution, while GitHub Secrets were used to securely store deployment-related variables and SSH authentication credentials required during CI/CD execution.

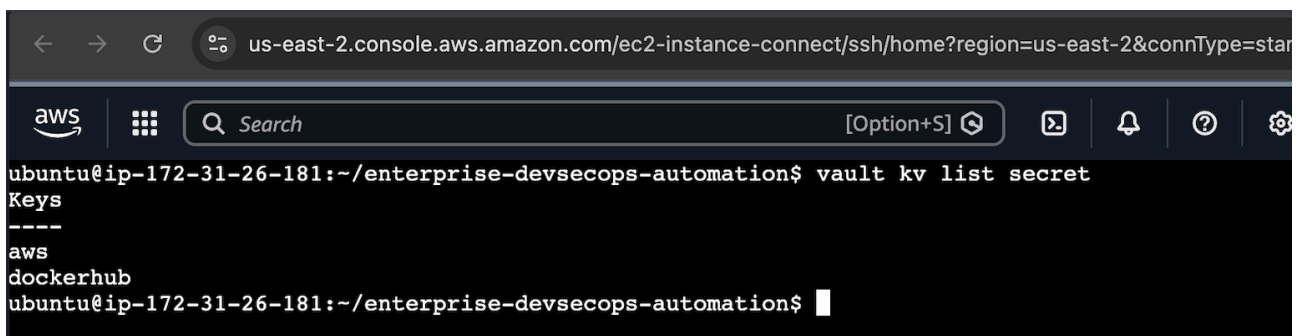
7.3.1 HashiCorp Vault Setup and Secrets Management

HashiCorp Vault was hosted on the Management EC2 instance and used to securely store sensitive credentials required by the pipeline.

Secrets Stored in Vault

Secret Name	Purpose
AWS_ACCESS_KEY_ID	Used for AWS authentication and Terraform provisioning
AWS_SECRET_ACCESS_KEY	Used for AWS infrastructure creation and management
DOCKERHUB_USERNAME	Used for Docker Hub authentication
DOCKERHUB_PASSWORD	Used for Docker image push authentication

Vault ensured that sensitive credentials were not hardcoded inside the application source code or CI/CD workflow files.



The screenshot shows a terminal window within the AWS Management Console. The terminal prompt is `ubuntu@ip-172-31-26-181:~/enterprise-devsecops-automation$`. The user has entered the command `vault kv list secret`. The output of the command is displayed as follows:

```
Keys
----
aws
dockerhub
ubuntu@ip-172-31-26-181:~/enterprise-devsecops-automation$
```

Vault Configuration Commands

Vault Startup Command

```
vault server -dev -dev-listen-address="0.0.0.0:8200"
```

Environment Variable Configuration

```
export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='*****'
```

Verification Commands

```
vault status
vault kv list secret
```

These commands were used to verify Vault server availability and validate stored secrets.

7.3.2 GitHub Secrets Management

GitHub Actions Secrets were used to securely store deployment-related variables and authentication credentials required during automated CI/CD execution.

These secrets allowed GitHub Actions workflows to securely interact with Vault and connect to the Deployment EC2 instance without exposing sensitive information inside workflow files.

GitHub Secrets Used

The following secrets were configured inside the GitHub repository:

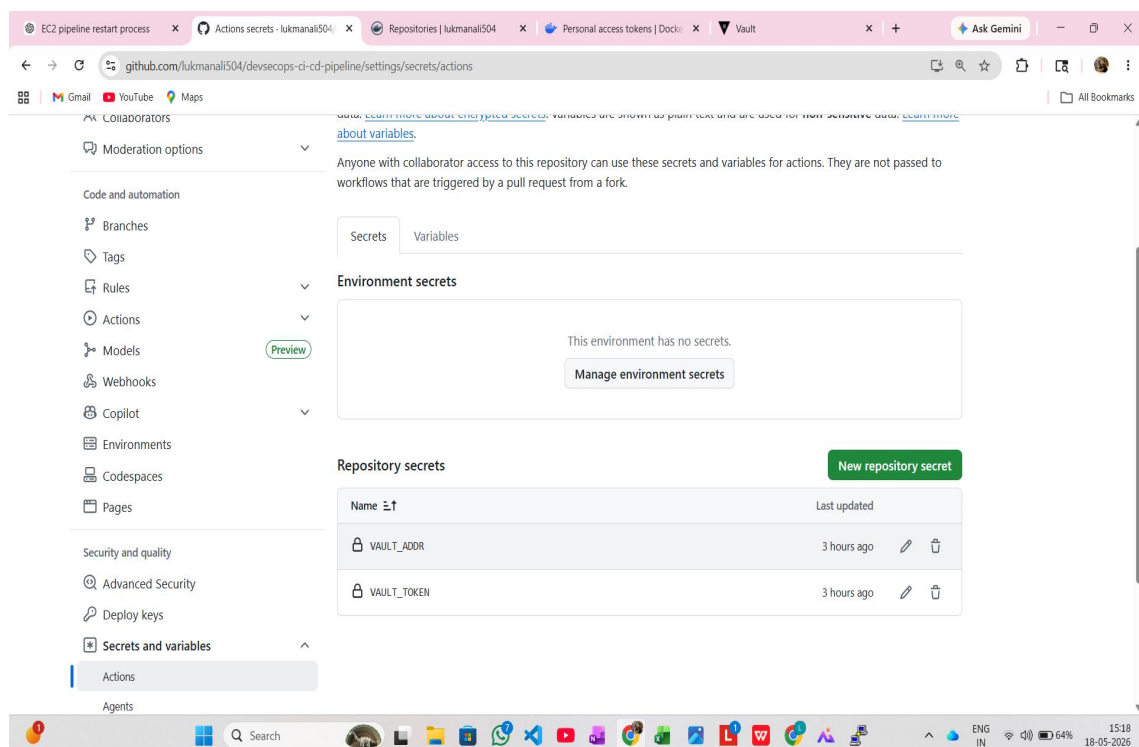
Secret Name	Purpose
VAULT_ADDR	Stores the Vault server address
VAULT_TOKEN	Used for Vault authentication

SSH Key Management

The SSH key pair used for deployment authentication was generated inside the Management EC2 instance.

The private SSH key was securely stored in GitHub Actions Secrets and used by GitHub Actions to establish secure SSH connections with the Deployment EC2 server during automated deployment.

This approach ensured secure authentication while maintaining CI/CD automation efficiency.



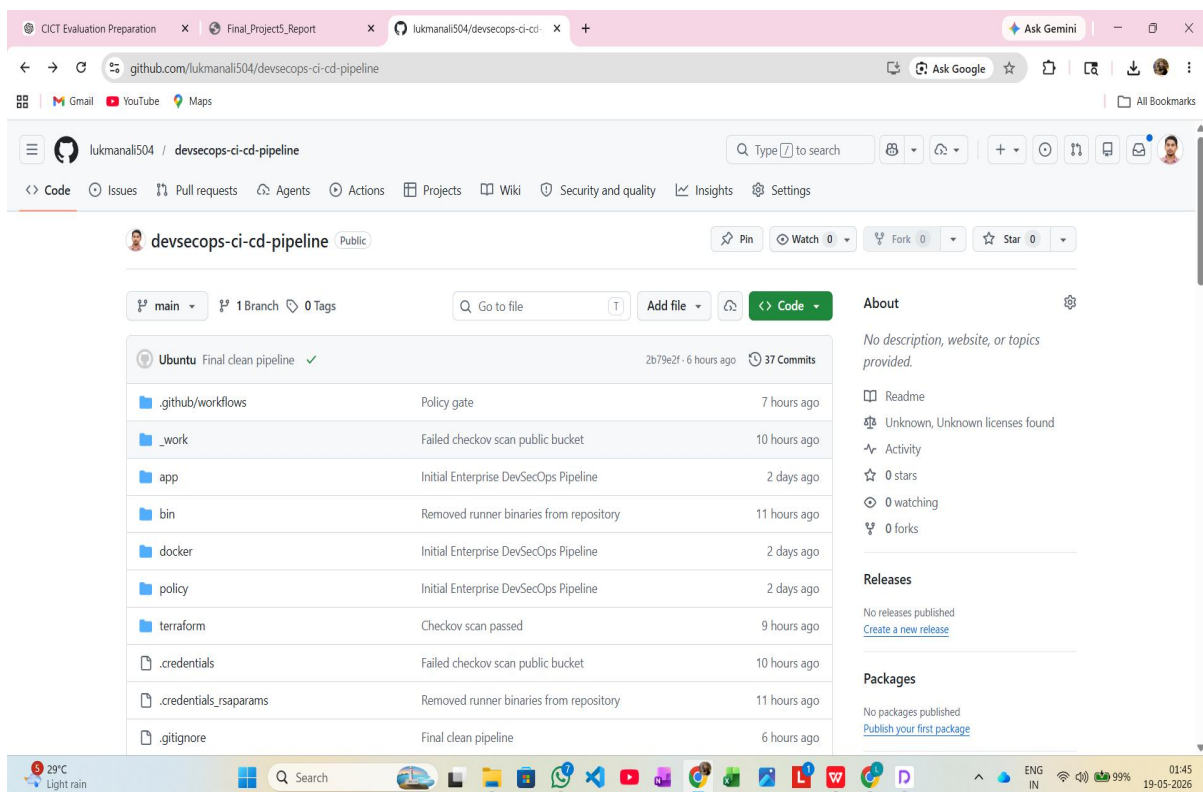
7.4 GitHub Repository Setup

The complete project source code was maintained in a GitHub repository.

The repository contains:

- Application source code
- Dockerfile
- Terraform configurations
- GitHub Actions workflow files
- Project documentation

GitHub provided centralized version control, branch management, and CI/CD workflow integration for the project.



7.5 GitHub Actions CI/CD Implementation

GitHub Actions was used to implement Continuous Integration and Continuous Deployment (CI/CD) automation for the project.

The workflow automatically triggers whenever code is pushed to the repository.

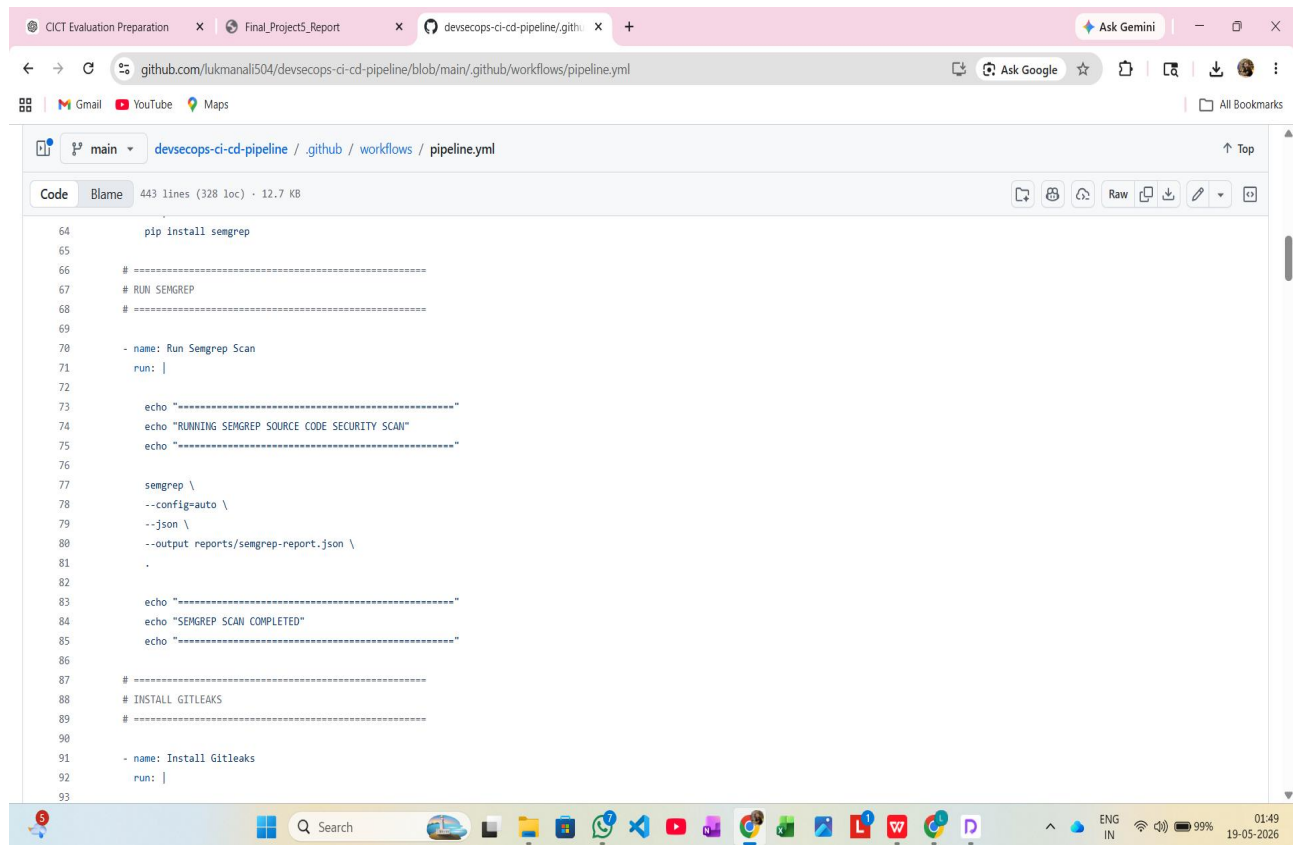
The pipeline performs:

- Security scanning
- Docker image build
- Docker Hub push
- Terraform infrastructure provisioning
- Automated deployment

Workflow File Location

.github/workflows/pipeline.yml

Figure: GitHub Actions Workflow Execution



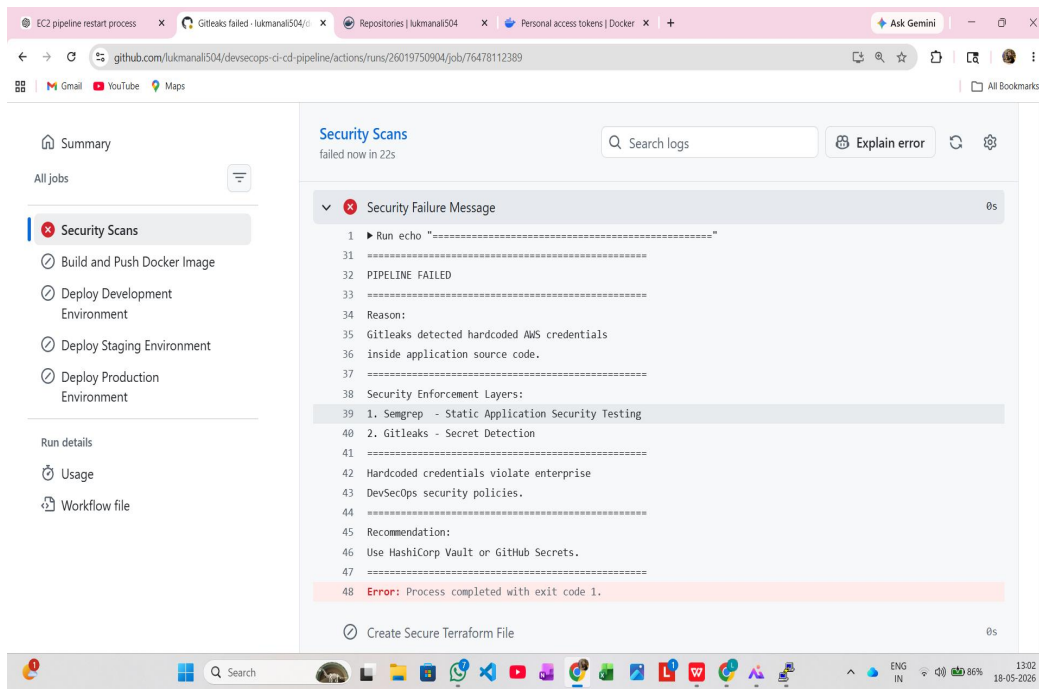
7.6 Security Tools Integration

Multiple security tools were integrated into the CI/CD pipeline to ensure continuous security validation before deployment.

Sengrep Static Application Security Testing

Sengrep was integrated into the CI/CD pipeline to perform Static Application Security Testing (SAST).

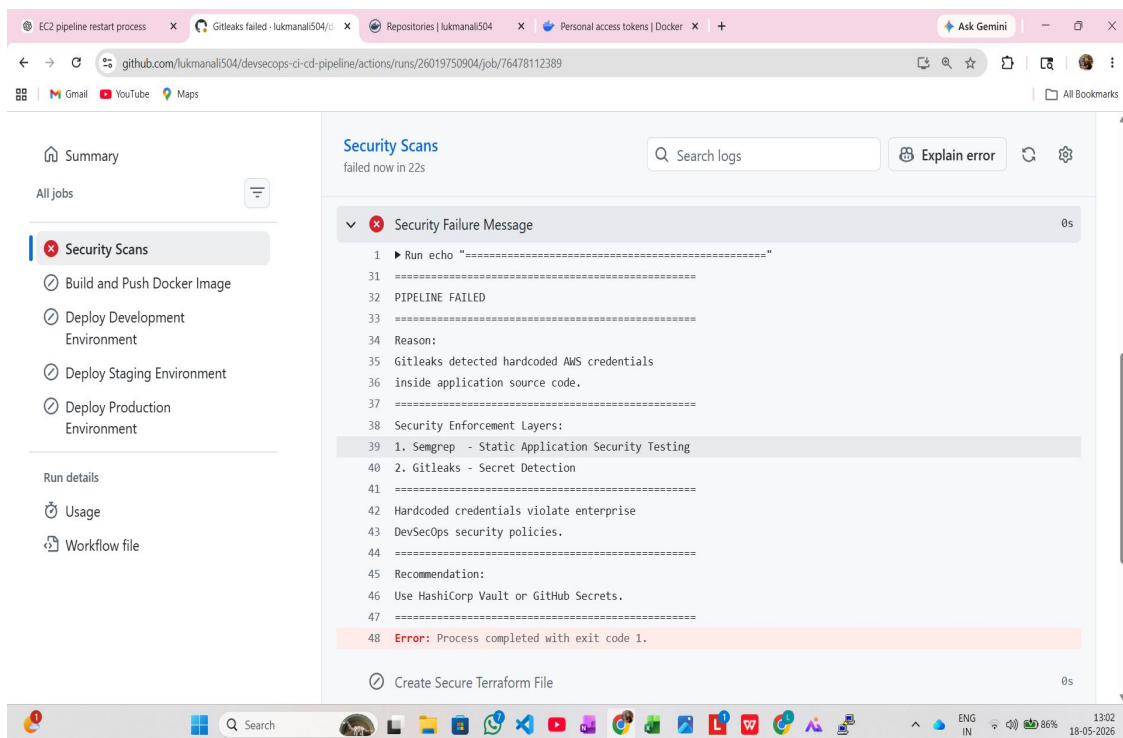
The tool analyzes source code for insecure coding practices, vulnerabilities, and risky configurations before deployment.



Gitleaks Secret Scanning

Gitleaks was integrated into the pipeline to detect hardcoded secrets such as API keys, passwords, and access tokens inside the source code repository.

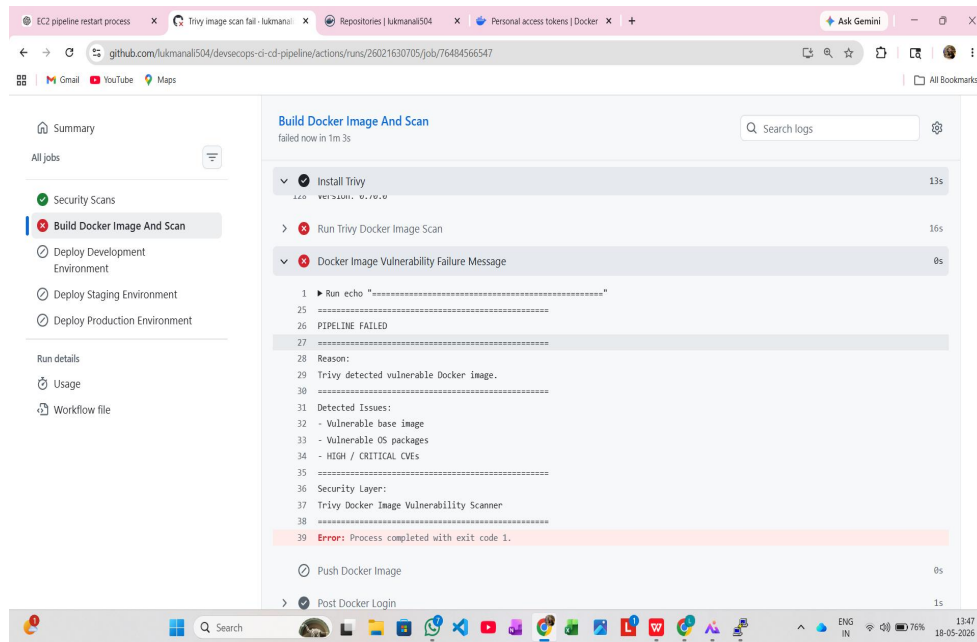
The tool helped prevent accidental exposure of sensitive credentials.



Trivy Vulnerability Scanning

Trivy was used to scan application dependencies and Docker container images for known vulnerabilities and security risks before deployment.

The pipeline was configured to block only CRITICAL vulnerabilities while still reporting HIGH severity findings.

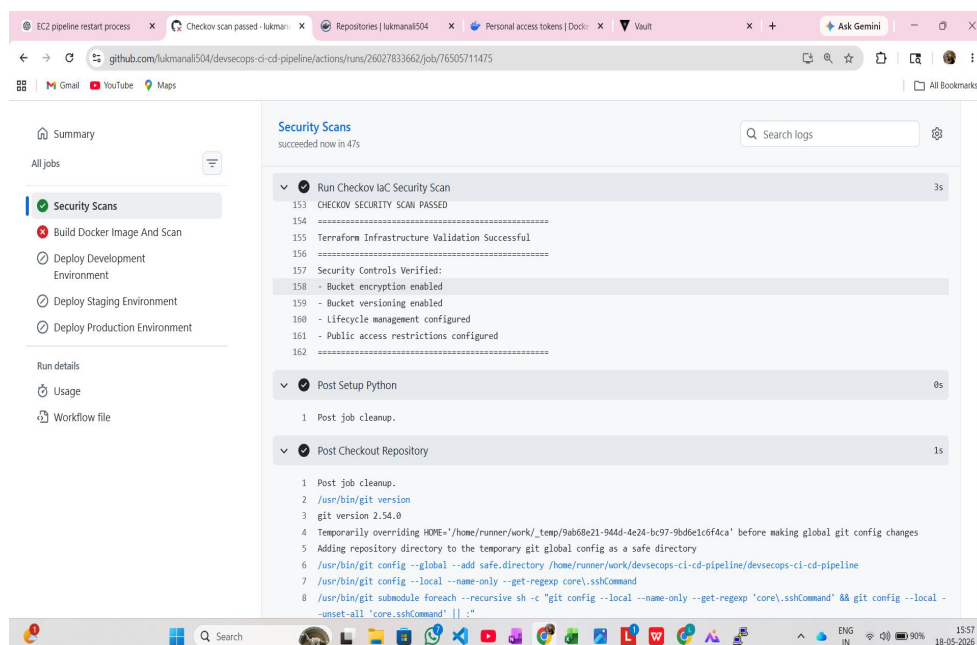


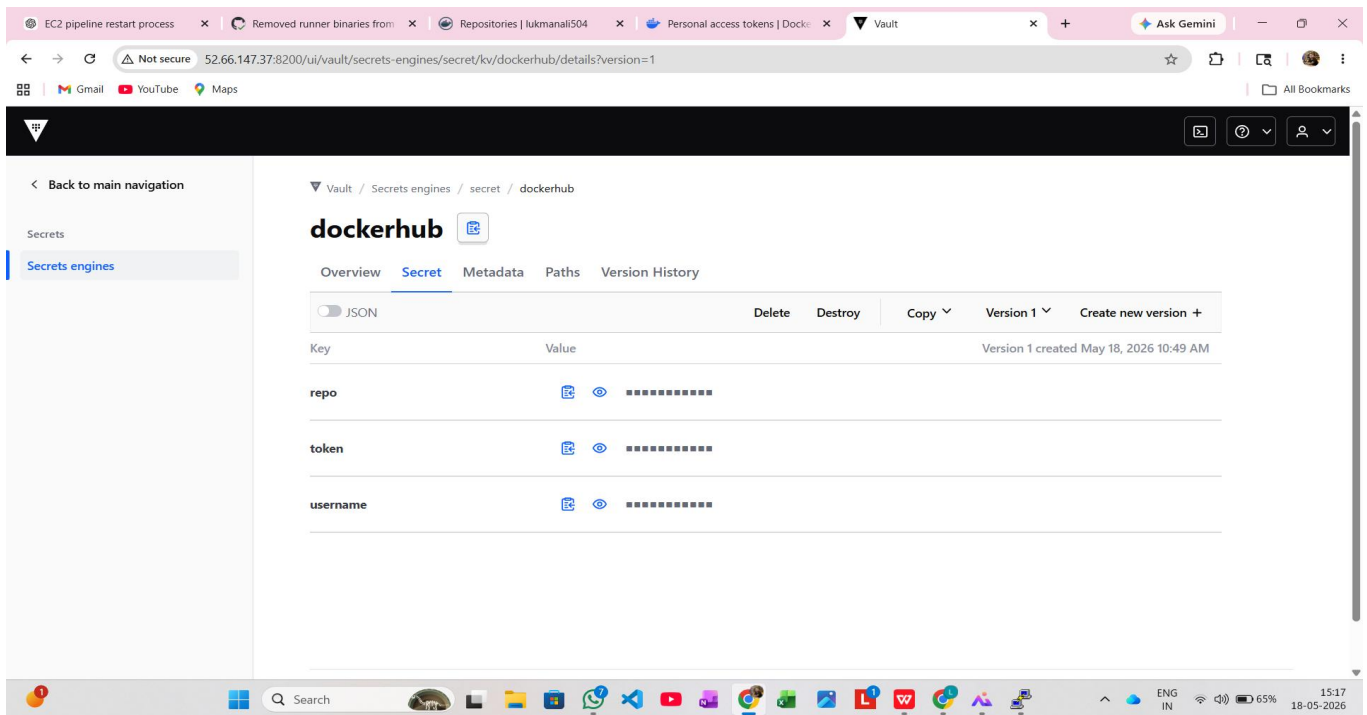
Checkov IaC Security Scanning

Checkov was integrated to perform Infrastructure as Code (IaC) security analysis on Terraform configuration files.

The tool validates AWS infrastructure configurations and identifies insecure cloud resource settings.

The Checkov integration ensured that security best practices were maintained during infrastructure provisioning and cloud deployment.





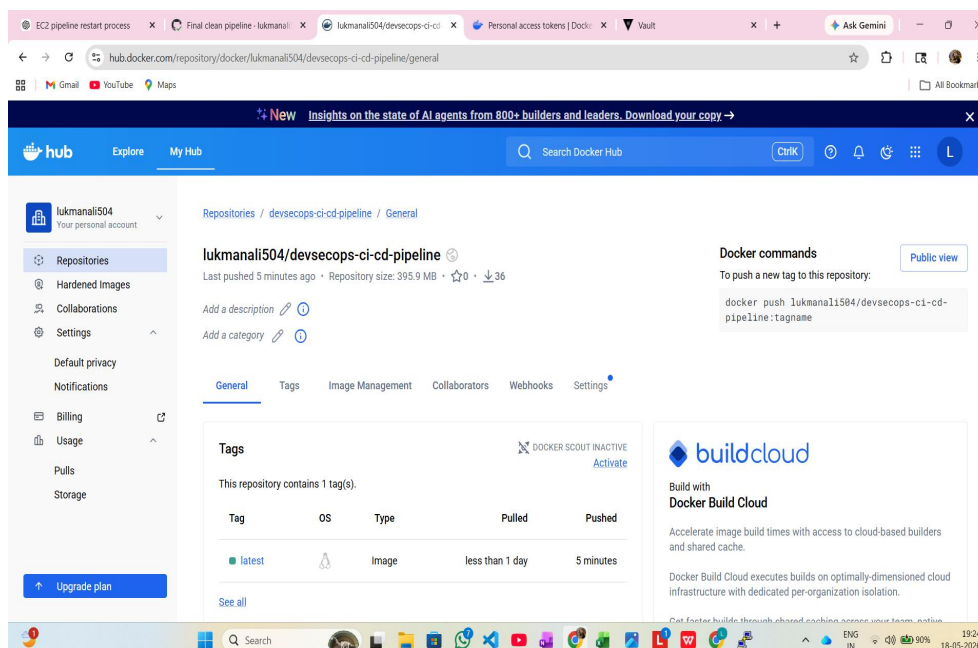
(DockerHub Credentials (Hashicorp Vault))

7.8 Docker Hub Integration

Docker images were pushed automatically to Docker Hub after successful pipeline execution. Docker Hub acted as the centralized container registry for storing deployment-ready Docker images.

Docker Hub Workflow

1. Build Docker image
2. Authenticate to Docker Hub
3. Push image automatically
4. Pull image during deployment



7.9 Deployment Workflow

The deployment workflow was fully automated using GitHub Actions.

GitHub Actions connects to the Management EC2 server through actions-runner and deploys updated Docker containers automatically.

The project implements multi-environment deployment to simulate an enterprise CI/CD release lifecycle.

Separate environments were created for Development, Staging, and Production using Docker container isolation.

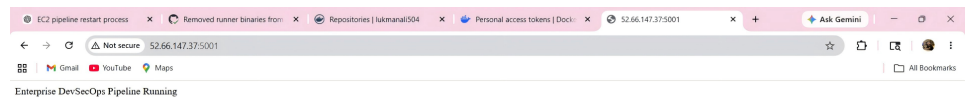
Each environment runs independently using separate Docker containers and port mappings.

Public Access URLs

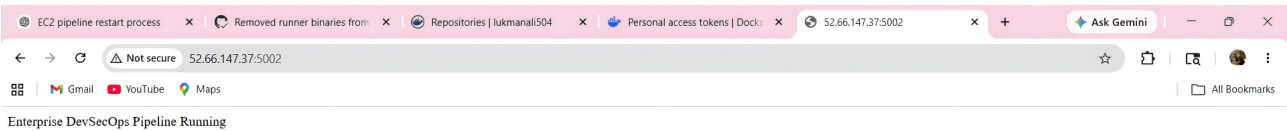
http://<public-ip>:5001
http://<public-ip>:5002
http://<public-ip>:5003

Environment Deployment Mapping

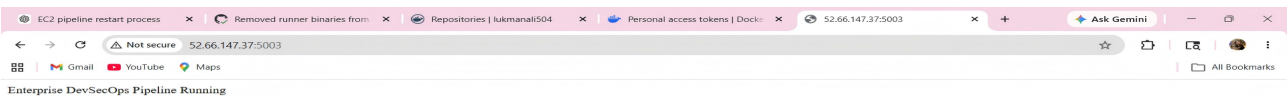
Environment	Port
Development	5001
Staging	5002
Production	5003



Dev



Staging



Prod

8. Challenges Faced During Implementation

During project implementation, multiple practical challenges were encountered and resolved.

8.1 Semgrep SAST Security Findings

Issue

Semgrep detected:

- Hardcoded credentials
- Insecure dependencies
-

Cause

The application initially lacked hardened security configurations.

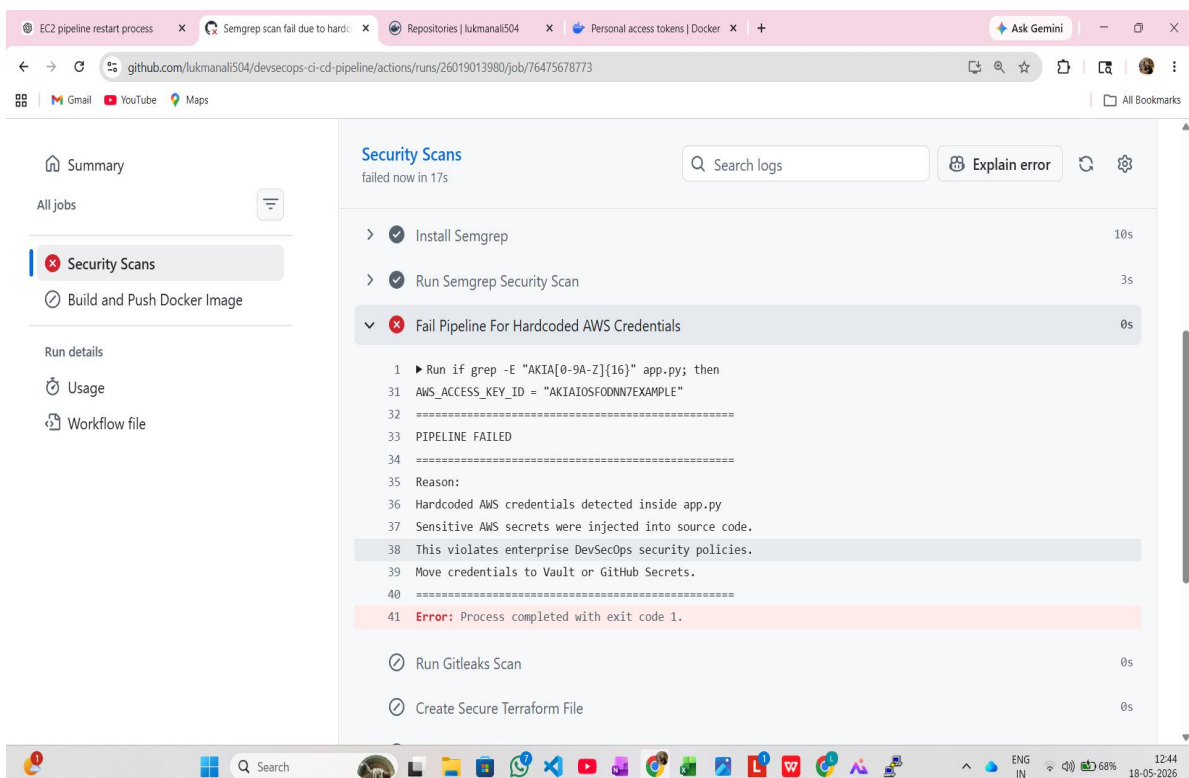
Solution Implemented

The following improvements were implemented:

- Added secure dependencies.
- Removed hardcoded credentials
-

Result

Security findings were reduced and pipeline stability improved.



8.2 Trivy Vulnerability Scan Failure

Issue

Trivy detected HIGH vulnerabilities inside app.py dependencies.

Example vulnerability:

- Installed Version: 7.0.3

Cause

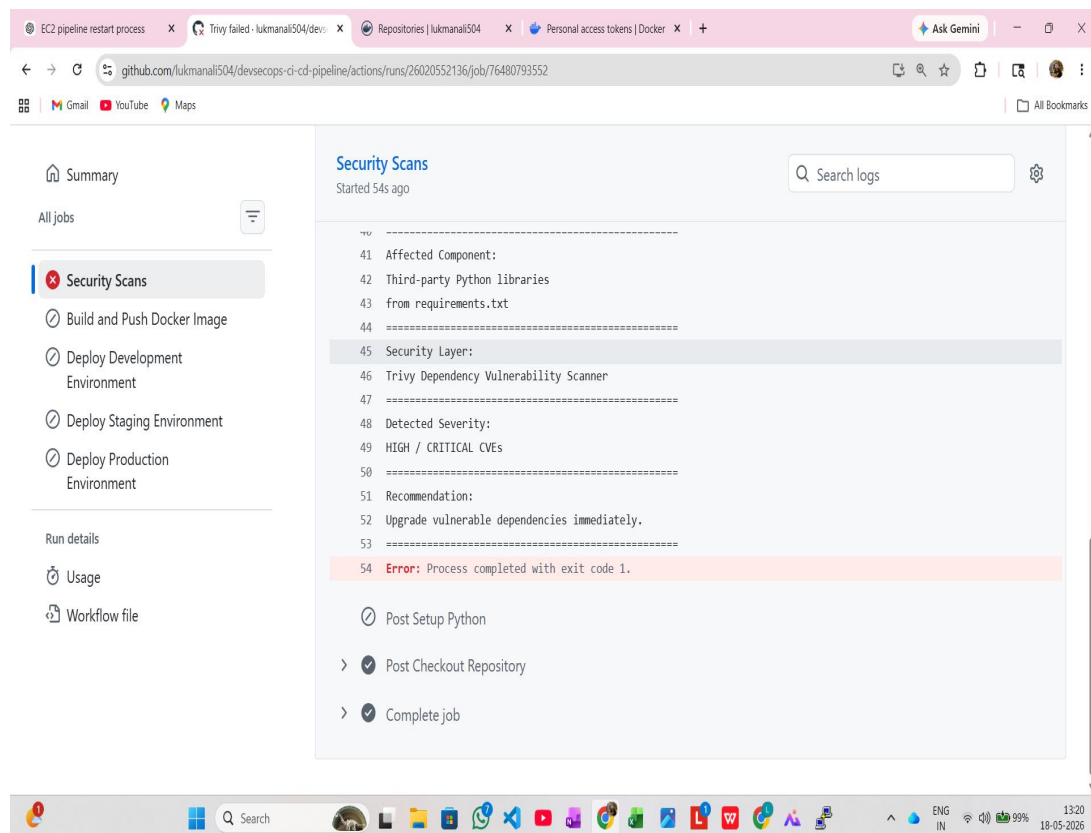
Vulnerable npm packages were present inside application dependencies.

Solution Implemented

The Trivy policy gate configuration was updated to block only CRITICAL vulnerabilities.

Result

- Pipeline stability improved
- Critical vulnerabilities continued to be blocked
- High vulnerabilities remained visible for monitoring



Trivy Scan Failed

8.3 Checkov IaC Security Scan Failure

Issue

Checkov detected multiple security findings in Terraform infrastructure configurations.

Security Issues Detected

1. Public AWS s3 bucket
public_acls= ["false"]
2. Publically accessible AWS s3 bucket objects exposed

Cause

SSH access and public ports were intentionally required for automated deployment and application accessibility.

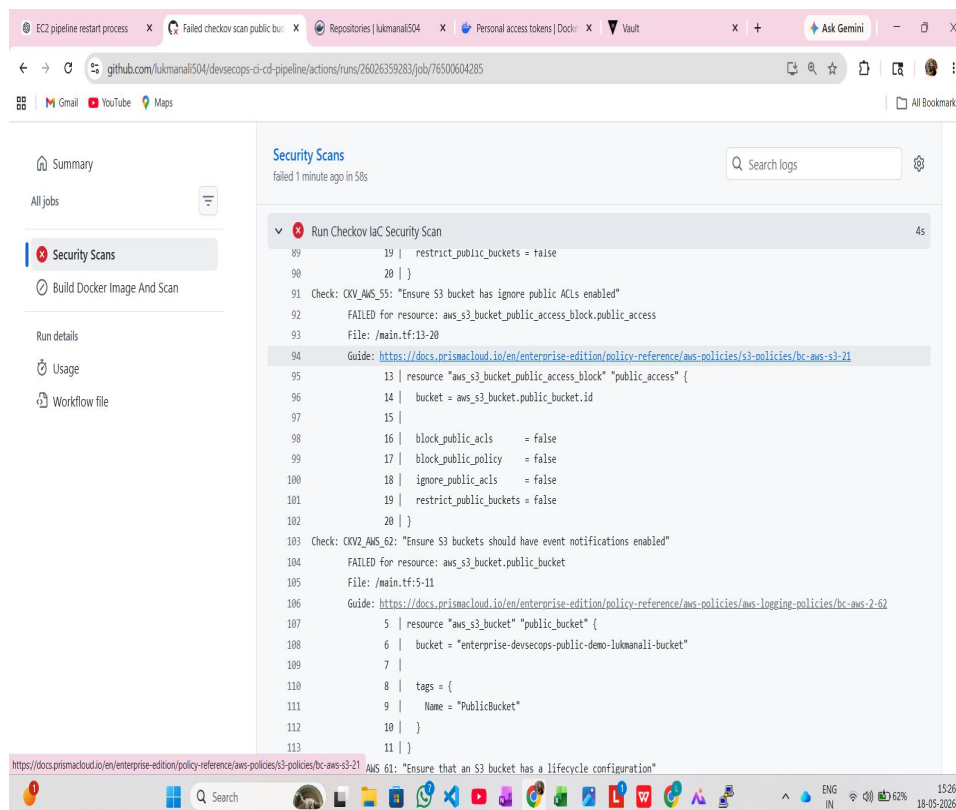
Solution Implemented

Configured:

block_public_acls="true"

Result

Pipeline execution continued successfully .



Checkov Scan Failed

8.4 Github actions-runner Configuration Issues

Issue

Deployment stuck due to job runner not found to run the deployment jobs.

Cause

Github actions-runner not configured properly on Management EC2 server.

Solution Implemented

- GitHub actions-runner configured properly.
- Added action-runners in .gitignore file.
- Correctly configured GitHub action-runner.

Result

Deployment jobs started properly and deployed .

9. Final Pipeline Execution Result

After resolving the various implementation challenges encountered during the project, the DevSecOps CI/CD pipeline executed successfully with all major stages completed successfully.

The issues related to:

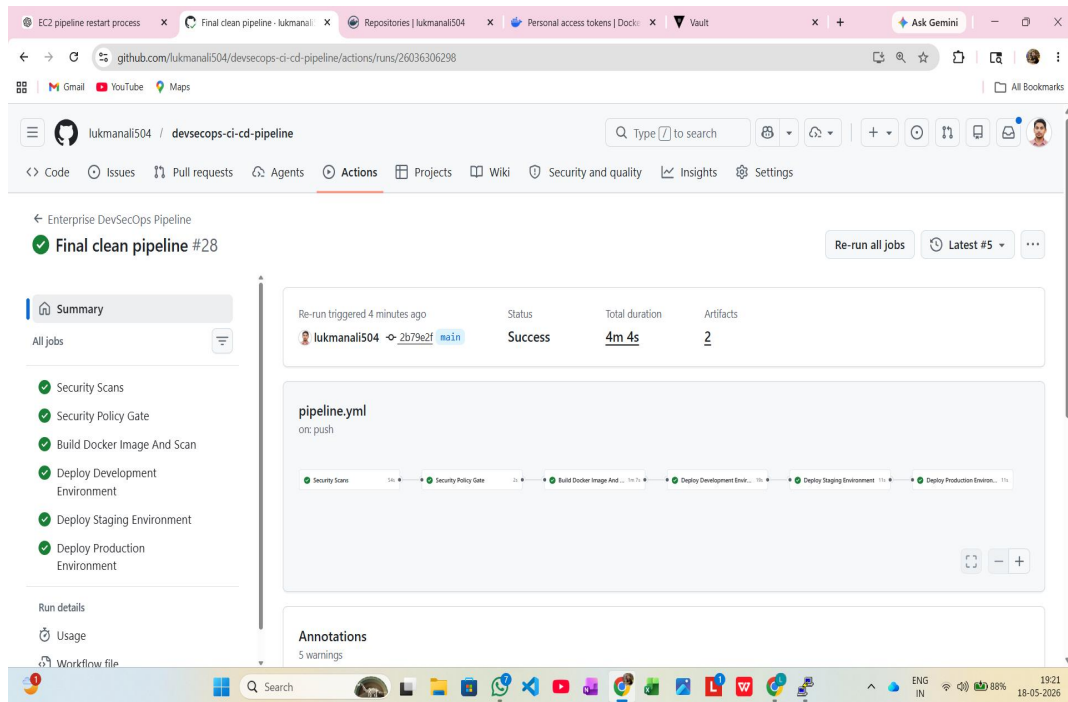
- Semgrep security findings
- Trivy vulnerability scanning
- Checkov IaC validation
- SSH deployment authentication

were analyzed and resolved during the implementation process.

As a result, the final pipeline successfully achieved:

- Secure CI/CD automation
- Automated infrastructure provisioning using Terraform
- Docker image build and deployment
- Secure secret management using HashiCorp Vault
- Continuous security validation
- Multi-environment container deployment
- Automated deployment using GitHub Actions

The final pipeline execution completed successfully with all required security checks, infrastructure provisioning stages, Docker deployment stages, and CI/CD workflow stages passing successfully.



(All Checks Passed)

10. Conclusion

The DevSecOps CI/CD Pipeline project successfully demonstrated the implementation of a secure and automated software delivery workflow by integrating development, security, and operations practices into a unified CI/CD pipeline.

The project automated the complete deployment lifecycle starting from source code integration to cloud deployment using GitHub Actions, Terraform, Docker, AWS EC2, HashiCorp Vault, and multiple security scanning tools.

Security was integrated throughout the pipeline using Semgrep, Gitleaks, Trivy, and Checkov to ensure secure application deployment and infrastructure provisioning.

Infrastructure as Code (IaC) automation using Terraform improved deployment consistency and enabled dynamic AWS infrastructure provisioning directly through the CI/CD pipeline.

Docker containerization ensured reliable and portable application deployment across Development, Staging, and Production environments.

HashiCorp Vault demonstrated centralized secret management practices by securely managing deployment credentials and sensitive configuration values.

Overall, the project successfully demonstrated practical DevSecOps concepts including CI/CD automation, cloud deployment, security integration, infrastructure automation, and multi-environment deployment management within a real-world deployment workflow.